Multi-Declaration Z Constructs — Q2(d) Demo

txt2tex

May 20, 2026

Background

The constructs in this file all share a single grammatical shape: a Z paragraph that introduces two or more typed variables in one declaration list, with a conjunction predicate over their projections, optionally followed by a characteristic expression. Until the Q2(d) parser fix, none of them parsed natively — students wrote them as raw LaTeX. They now compile to fuzz-clean output.

$$[ShipName, ClassName, BattleName, CountryName, Result]$$

Class

$$class : ClassName$$
$$country : CountryName$$
$$numGuns : \mathbb{N}$$
$$bore : \mathbb{N}$$
$$bore \leq 16$$

PK(Class) = $\{class\}$

Ship

$$name : ShipName$$
$$class : ClassName$$

PK(Ship) = $\{name\}$

Outcome

$$ship : ShipName$$
$$battle : BattleName$$
$$result : Result$$

PK(Outcome) = $\{ship, battle\}$

1 . WYSIWYG line breaks for algebra operators

Long relational-algebra chains can be broken at any operator. Natural newlines work; explicit \ continuation also works. Display position wraps in array; the chain reads top-to-bottom instead of running off the page.

$$\begin{array}{l} \text{Project}\{name, country\}(Class \otimes \\ \quad Ship \otimes \\ \quad \text{Rename}_{ship \to name}(\text{Project}\{ship\}(\text{Restrict}_{battle = \text{`}Guadalcanal\text{'}}(Outcome)))) \end{array}$$

2 . Multi - decl comprehension with conjunction predicate

This is the canonical multi-typed relational-calculus form for any query that crosses two or more relations. Before the fix it required a raw LATEX block; now it parses natively.

$$\{ s : Ship;\ c : Class;\ o : Outcome \mid s.class = c.class \land o.ship = s.name \land o.battle = \text{`}Guadalcanal\text{'} \bullet \langle name == s.name, country == c.country, bore == c.bore \rangle \}$$

3 . Multi - decl quantifier with conjunction predicate

‘$\forall\, s : T;\ c : U \bullet predicate.$ body‘ works the same way — nested Quantifier nodes covering the chained declarations.

$$\forall\, s : Ship;\ c : Class \mid s.class = c.class \land c.bore \geq 16 \bullet c.numGuns > 8$$

4 . Multi - decl quantifier without characteristic expression

When the predicate alone suffices — no ‘.‘ separator, no body expression — the comprehension defaults the characteristic tuple to the signature (Z RM §3.9).

$$\{ s : Ship;\ c : Class \mid s.class = c.class \land c.bore \geq 16 \}$$

5 . Multi - decl mu (definite description)

‘mu‘ parses with multi-decl signatures the same way. The current generator emits nested ‘$(\mu ...| (\mu ...))$‘ $-$ $-$ $-semanticallyequivalentbutnotSpivey-canonical.Thesamespecial-casetreatmentlambdajustreceivedisafollow-upformu.Single - declmu(shownhere)emitscleanly.$

$$bigGun == (\mu\, c : Class \mid c.bore = 16 \bullet c.numGuns)$$

6 . Multi - decl lambda — Spivey - canonical form

Lambdas of more than one variable now emit as a single $\lambda\, tokenwithsemicolon-joinedSchemaText, matchingZRM3.12andth$

$$fleet == (\lambda\, s : Ship;\ c : Class \mid s.class = c.class \bullet s.name)$$

7 . Paren - wrap fix

Fuzz requires ‘$(\lambda ... \bullet ...)$‘$---parenthesesaroundeverylambdaexpression, atanyposition.Thegeneratorpreviouslyonlywrappe$ $levelzedpredicatesemittedbare‘\ \lambda\ ‘, whichfuzzrejectedwithOpening parenthesis expected at symbol \lambda . The fix : drop the ‘paren$ $(\lambda\, x : \mathbb{N} \bullet x + 1)$